them at groups of clusters by labels.

Argo CD can also work at scale. ApplicationSets let you template deployments across clusters.

Akri exposes IoT devices as Kubernetes resources that pods can discover. Useful when you have a mix of workloads and physical devices at each site.

For my friend's chain, Rancher plus Argo CD worked well. The ops team got a central dashboard. Developers worked with Git. When a new store opened, provisioning a cluster was a two-hour job instead of a week.

Storage, networking, and the things that bite

Storage is weirder at the edge. In the cloud, you have managed block and object storage. At the edge, you have the local disk, which is often slow, sometimes small, and occasionally unreliable. For data that matters, think about what happens when the SD card dies.

Networking at the edge means dealing with NAT, firewalls, and sites that cannot be reached from outside. For reaching into edge sites from the cloud, WireGuard is popular and lightweight. Tailscale builds on WireGuard with easier key management.

Resource contention is real. Edge devices often run more than Kubernetes workloads. Set resource requests and limits carefully. Use node labels and taints to control what runs where.

Updates become a bigger deal. Updating a Kubernetes version in a data centre is routine. Updating it on three thousand edge devices is a project. Plan for staged rollouts and test on a canary site first.

Security at the edge is different

Physical access becomes a real threat. Somebody can walk up to the device. So encrypt disks. Use secure boot if the hardware supports it. Assume local credentials can be extracted and limit what they grant.

Network trust is weaker. Every connection should be authenticated and encrypted. Zero trust principles apply even more strongly.

Supply chain risk is higher. Images should be signed. Admission controllers should enforce signed images only. Compromise of one edge site should not compromise others.

Secrets management gets tricky. You cannot assume *Vault* is always reachable. Cache secrets locally with care or use hardware security modules on the device.

A real-world example, condensed

Here is what my friend's retail setup looked like after we finished the migration.

Each store had an Intel NUC with eight gigabytes of RAM running K3s. The cluster ran five workloads -- the point-of-sale backend, the inventory sync agent, the customer display service, a tiny Prometheus instance, and a log forwarder shipping to the central platform.

All configurations lived in Git. Argo CD on each cluster pulled updates every two minutes. Deployments were pull requests. Developers never touched a store directly.

The cloud side had Rancher managing all forty clusters, Loki aggregating logs, and Grafana Cloud showing fleet-wide dashboards. Connectivity ran over WireGuard tunnels. Secrets came from a *Vault* instance each store cached locally.

When a new store opened, the NUC was imaged with a prebuilt K3s installer including the store's identity and GitOps config. Plug it in, and within fifteen minutes the store had a running cluster syncing from Git. No DevOps engineer was required on site.

Where this is heading

A few trends are worth watching.

- eBPF is making serious inroads at the edge. Cilium works well as a CNI on resource-constrained devices.
- WebAssembly is starting to look promising for edge workloads. Smaller than containers, faster startup, more portable. Projects like SpinKube bring WASM workloads into Kubernetes.
- AI at the edge keeps growing. Running ML inference close to data sources is becoming routine.
- Fleet management tooling keeps improving. Platforms like Rancher Prime and Portainer Edge are competing on how well they handle thousands of distant clusters.

The quiet revolution of orchestration everywhere

Not long ago, the cloud was where modern operations happened. Everything else was legacy. Edge computing changes that story. The same tools, the same patterns, the same deployment workflows now reach into physical spaces that used to be unreachable. The petrol pump. The retail store. The factory floor. The hospital ward. For teams that already know Kubernetes, this is a huge unlock. The learning curve is manageable. The tools are open source. The payoff is a drastically simpler way to run software across many locations. If you have any workload that currently requires somebody physically visiting a site to update, you should probably be looking at this seriously. Start small. Pick one site. Install K3s. Deploy one workload. The pattern scales from there. END

By: Dhaval Gajjar

The author is the CTO of Textdrip and the CEO of Pranshtech. He spends his days building and scaling production systems and writes occasionally about the messy, unglamorous side of keeping services alive.